Projektuppgift

DT211G, Frontend-baserad webbutveckling

Projektarbete
Vibe-Caster

Frida Pihlström

Mittuniversitetet
MID SWEDEN UNIVERSITY

Campus Härnösand Universitetsbacken 1, SE-871 88. **Campus Sundsvall** Holmgatan 10, SE-851 70 Sundsvall.
Campus Östersund Kunskapens väg 8, SE-831 25 Östersund.
Phone: +46 (0)771 97 50 00, Fax: +46 (0)771 97 50 01.

MITTUNIVERSITETET
Avdelningen för informationssystem och -teknologi

Författare: Frida Pihlström, frpi2500@student.miun.se
Utbildningsprogram: Webbutveckling, 120 hp
Huvudområde: Datateknik
Termin, år: VT, 2026

Sammanfattning

Med det här projektet har jag byggt ”Vibe-Caster”, det är en webbapplikation som automatisk matchar musik efter det lokala vädret. Genom att kombinera OpenWeatherMap, Leaflet och Deezer skapas en ”mashup” där användaren kan söka på en stad och få en känsla för stämningen på platsen. Arbetet har genomförts med hjälp av verktyg som Vite för automatisering och Sass för styling. Resultatet blev en responsiv webb-app med interaktiva kartor, en inbäddad musikspelare och väderstyrda animationer. Slutsatsen av projektet är att API-integration ger väldigt stora möjligheter att skapa unika användarupplevelser, men också att de kräver mycket felhantering.

1 Innehållsförteckning

Sammanfattning iii
2 Introduktion / Inledning 1
2.1 Bakgrund och problemmotivering 1
2.2 Övergripande syfte 1
2.3 Avgränsningar 1
2.4 Konkreta och verifierbara mål / Detaljerad problemformulering 1
2.5 Översikt 2
2.6 Författarens bidrag 2
3 Teori / Bakgrundsmaterial 3
3.1 API och JSON 3
3.2 Mashup-konceptet 3
3.3 Automatisering med Vite 3
3.4 Sass och SCSS 3
3.5 Kartvisualisering, Leaflet och OpenStreetMap 3
3.6 Asyncron JavaScript (Promises och Async/Await) 4
3.7 CORS och Proxy-lösning 4
3.8 Documentation med JSDoc 4
4 Metod 5
4.1 Arbetsprocess och verktyg 5
4.2 API-hantering och asyncron kod 5
4.3 Felsökning och robusthet 5
4.4 Styling och responsiv design 6
5 Konstruktion / Lösningsalternativ 7
5.1 getMusicVibe ..... **Error! Bookmark not defined.**
5.2 Dataflöde mellan API:er 7
5.3 Design och visuell profil 7
6 Resultat 9
6.1 Appens utseende och flöde 9
6.2 Väderstyrda teman och animationer 9
6.3 Karta och interaktivitet 10
6.4 Musikintegration och Preview 10
6.5 Responsiv design i praktiken 10
6.6 Teknisk dokumentation 11
7 Slutsatser / Analys / Diskussion 12
Källförteckning 13

2 Introduktion / Inledning

Ibland när man lyssnar på musik så tittar man ut genom fönstret en regnig dag och känner att musiken man lyssnar på inte alls stämmer överens med känslan som vädret ger. Det var det jag ville lösa genom idén med Vibe-Caster. Jag ville bygga något som hittar den perfekta musiken som matchar ”viben” utomhus.

2.1 Bakgrund och problemmotivering

När jag började fundera på vad jag skulle göra med det här projektet ville jag göra något som kändes användbart, gick igenom massa idéer om huruvida det skulle vara informativt, kul, kreativt eller tokigt. Det finns så oändligt många API:er att välja från. Så jag utgick från mig själv och tänkte att jag gillar att lyssna på musik, men ofta blir det bara samma låtar om och om igen. Därför ville jag ta fram något som kunde ge musiktips. När jag satt och funderade, så tittade jag ut, såg att det var grått och tråkigt (som det ofta är i Skåne), och då slog det mig att det skulle vara kul att ha musik som bestäms av hur vädret är för tillfället. Så då föddes idén av att låta väderdata, från en vald plats på kartan, kan styra musiken.

2.2 Övergripande syfte

Syftet med mitt projekt är egentligen att testa vad man kan göra genom att låta olika API-tjänster prata med varandra, alltså att göra en ”mashup” [1]. Jag ville se om jag kunde bygga en sorts automatiserad dj som anpassar sig efter vädret och humöret som kommer med vädret. Istället för att jag ska behöva sitta och leta fram musik, så ville jag att koden skulle göra det åt mig.

2.3 Avgränsningar

Att hålla mig från att lägga till fler API:er var svårt, men jag valde att hålla mig till tre huvuddelar: väder, karta och musik. Jag har inte lagt till något sätt att spara sina favoritlåtar på eller generera nya spellistor. Man kan inte heller spara en plats.

2.4 Konkreta och verifierbara mål / Detaljerad problemformulering

Mina mål med projektet var ganska tydliga när jag väl hade kommit på idén som jag ville köra på.

- Hämta väderdata i JSON-format från OpenWeatherMap [2].
- Visa platsen man har sökt på med en interaktiv karta från Leaflet [3].
- Jag ska bygga en funktion som väljer ut rätt spellista från Deezer [4].
- Sidan ska vara responsiv.

- Jag ska använda Vite [5] för att hålla ordning på allt och se till att sidan laddar snabbt.

- Jag ska använda Sass för att kunna jobba smidigare med färgerna och designen.

2.5 Översikt

Rapporten är uppdelad i sex kapitel. I kapitel 2 går jag igenom teorin och en del tekniska begrepp, som API:er och Sass för att förklara vad jag har använt för att bygga appen/sidan. Kapitel 3 beskriver min metod och hur jag har jobbat med utvecklingen. I kapitel 4 går jag in på själva konstruktionen, hur den är uppbyggd och hur jag löste designen. Kapitel 5 visar det färdiga resultatet och slutligen i kapitel 6 reflekterar jag över vad jag har lärt mig och vad jag hade kunnat göra annorlunda.

2.6 Författarens bidrag

Jag har planerat, designat och programmerat hela applikationen själv. Arbetet har innefattat allt från research av olika API:er och sen uppsättning av utvecklingsmiljö i Vite, styling med Sass samt implementation av asynkron JavaScript.

3 Teori / Bakgrundsmaterial

Här går jag igenom lite av tekniken jag har använt för att få allt att fungera.

3.1 API och JSON

Ett API (Application Programming Interface) fungerar som en länk mellan en app och de stora tjänsterna. När man frågor till exempel OpenWeather om vädret, så svarar de med data i ett format som kallas JSON. Det ser ut som en lång lista med information som sedan JavaScript kan läsa av och plocka ut just det jag ber om, som till exempel temperatur eller huruvida det regnar eller är klart.

3.2 Mashup-konceptet

Att bygga en mashup handlar väl främst om att vara lite kreativ med redan färdig data. Man ska integrera funktioner från flera olika källor och få dem att samexistera eller samarbeta. I mitt fall betyder det att min vädertjänst bestämmer vad musiktjänsten ska göra.

3.3 Automatisering med Vite

För att slippa sitta och ladda om sidan manuellt vid varje ändring, och för att optimera alla filer, så har jag använt Vite [5]. Det är ett verktyg som sköter själva byggandet av webbplatsen automatiskt, så att jag kan fokusera på att skriva koden.

3.4 Sass och SCSS

För att styla sidan valde jag Sass istället för vanlig CSS. Sass låter en använda funktioner och påminner lite mer om programmering i JavaScript då man kan använda variabler och nesting. Enligt dokumentationen för Sass [7] gör det att man kan skriva mer lätthanterlig kod.

Det jag upplever är bäst med Sass är att jag kan dela upp stylingen i mindre moduler och därmed slipper man ha en CSS-fil som har hundra- eller tusentals rader. Det sparar faktiskt väldigt mycket tid att bara behöva ändra färgen på en plats och inte leta upp alla platser där färgen man vill byta ut finns med.

3.5 Kartvisualisering, Leaflet och OpenStreetMap

För att visa kartan använde jag biblioteket Leaflet [3]. Det är ett ”open source”-bibliotek för interaktiva kartor. Leaflet är verktyget som visar kartan och sätter ut prickar, men själva kartorna kommer från OpenStreetMap.

Kartan fungerar genom att ladda in ”tiles” som pusslas ihop beroende på vilka koordinater man har skickat in. Leaflet är gjort för att fungera bra på mobiler vilket underlättar när jag ska göra en responsiv design.

3.6 Asynkront JavaScript

En viktig del I projektet är hur JavaScript hanterar väntetider. Man kan inte veta när man anropar ett API hur lång tid det kommer ta att få svar. För att sidan inte ska ”frysa” under tiden man väntar på svar så används asynkron programmering.

Genom att använda async och await säger man till koden att vänta tills den får svar, men låta resten av sidan laddas in så länge. Detta bygger på promises [8] eller löften om att data kommer levereras.

3.7 CORS och Proxy-lösning

CORS (Cross-Origin Resource Sharing) är en säkerhetsinställning i webbläsaren. Den finns till för att stoppa skadliga skript att hämta data utan tillåtelse [6].

Deezers API, som säkert många andra, är inställd på att bara svara om anropet kommer från en ”godkänd” server och inte direkt från en lokal dator. Detta problem stötte jag på under utvecklingen och kommer beskriva mer i nästa kapitel (Metod).

3.8 Dokumentation med JSDoc

För att hålla ordning på mina funktioner och göra det lättare att gå tillbaka i framtiden och minnas vad koden gör använde jag JSDoc. Det är en standard för att skriva kommentarer direkt i koden som förklarar vad en funktion tar emot för data och vad den sen skickar med tillbaka.

4 Metod

När jag satte igång med projektet så valde jag att arbeta stegvis, och inte försöka bygga upp allt på en gång.

4.1 Arbetsprocess och verktyg

Jag började med att sätta upp projektet i Vite. Det var verkligen något som underlättade under arbetet eftersom sidan uppdateras automatiskt så fort en fil sparas och man kan direkt se om ändringarna funkade eller inte. För att hålla ordning på allt använde jag Git och försökte göra commits hyffsat regelbundet för att kunna gå tillbaka om jag råkade förstöra något.

4.2 API-hantering och asynkron kod

Det viktigaste med appen är ju att hämta data från de olika tjänsterna. Där använde jag mig av fetch och asynkron programmering med async och await. Det var lite klurigt i början att få ordning på flödet. Jag insåg snabbt till exempel att jag var tvungen på att vänta på svar från OpenWeatherMap innan jag kunde göra något annat, eftersom både kartan och musiken helt beror på vilken stad och väder det handlar om.

En utmaning jag stötte på var när jag skulle hämta data från Deezers API. Webbläsaren stoppade anropen på grund av något som kallas CORS (Cross-Origin Resource Sharing) [6]. Efter att ha läst och kollat en del YouTube, så kom jag fram till en lösning. Det var att använda en proxy-server(cors-anywhere) som fungerar som en mellanhand för att kunna använda Deezer i min lokala utvecklingsmiljö.

4.3 Felsökning och robusthet

Under arbetets gång märkte jag att sidan ibland kraschade när ingen spellista kunde hittas som matchade. Jag la därför till en ”fallback-lösning”. Om min första sökning inte ger något så kör koden istället en säker sökning på ”top hits 2026”. På så sätt blir sidan stabilare för besökare.

Jag fick också jobba en del på variabelordningen. Jag fick ett ReferenceError ett par gånger för att jag försökte skicka med väderinformation till kartan innan informationen hade sparats ner i en variabel. Så jag fick ändra om lite grann i vilken ordning funktionerna skulle köras för att sidan skulle bli hållbar.

När jag försökte publicera sidan på Netlify så fick jag fel-meddelanden som hade att göra med CORS-problematiken igen. Det ledde till att jag fick spendera en hel del tid med Google och YouTube igen, och till hittade jag en lösning med netlify.toml-fil som jag skapade för att vara som en tunnel mellan klienten och Deezers API.

Även Leaflet-ikonerna på kartan försvann vid publicering, men jag hittade en lösning i att länka direkt till ikonerna.

4.4 Styling och responsiv design

För att få till den grafiska profilen använde jag Sass (SCSS). Jag valde att dela upp den i flera moduler: _reset.scss för att nollställa webbläsarens inställningar, _base.scss för typsnitt och färger och _layout.scss för själva strukturen på sidan. Det gjorde det mycket lättare att hålla koll på CSS:en och istället för att scrolla runt i en lång, lång lista så kunde jag direkt gå till rätt fil.

När jag testade sidan på min mobil märkte jag att musikspelaren ibland stack ut utanför skärmen. Jag gick då tillbaka och ändrade mina fasta mått till procentuella mått och använde en ”max-width” för att det inte skulle bli för utdraget på större skärmar.

5 Konstruktion / Lösningsalternativ

I det här kapitlet går jag igenom hur jag faktiskt har byggt ihop allting. Jag har försökt hålla koden så tydlig som möjligt genom att dela upp den i mindre delar som sköter sina egna uppgifter.

5.1 Musik-funktion

Det som gör Vibe-Caster unik är hur den väljer ut musik. Istället för att bara slumpa fram en spellista skapade jag en funktion som heter getMusicVibe. Den tar emot två saker från väder-API:et och det är väderförhållandet (till exempel ”Rain” eller ”Clear”) och temperaturen.

Jag använde vanliga if/else-satser för att para ihop vädret med rätt sökord för Deezer. Till exempel:

- Om det är soligt (Clear) och över 15 grader, söker den på ”Summer Party”.
- Om det regnar (Rain) och är kallt, söker den på ”Melancholic Piano”.
- Om det åskar (Thunderstorm) söker den efter ”Dramatic Dark Techno”.

5.2 Dataflöde mellan API:er

En av de stora utmaningarna var att få data att gå smidigt mellan de tre tjänsterna. Processen ser ut så här:

1. Sökning – Användaren skriver in en stad.
2. Väder – Appen hämtar data från OpenWeatherMap. Härifrån får jag koordinater (latitud och longitud), temperatur och vädertyp.
3. Karta – Koordinaterna skickas direkt till Leaflet-funktionen som flyttar kartan och sätter ut en markering.
4. Musik – Vädertypen och temperaturen skickas till getMusicVibe som returnerar ett sökord. Det sökordet används för att göra ett nytt anrop till Deezer.

5.3 Design och visuell profil

För designen ville jag hålla det modernt, men ändå mjukt. Jag ville testa att använda Glassmorphism [8]. Det innebär att man använder halvgenomskinliga bakgrunder och lägger på en suddig effekt för att efterlikna frostat glas.

Jag gjorde ett par skisser först på hur jag ville att sidan skulle kännas. Jag visste ju att det skulle finnas en sökruta, en karta och en musikspelare och att designen skulle hållas enkel så det blev bara några snabba skisser i Figma.

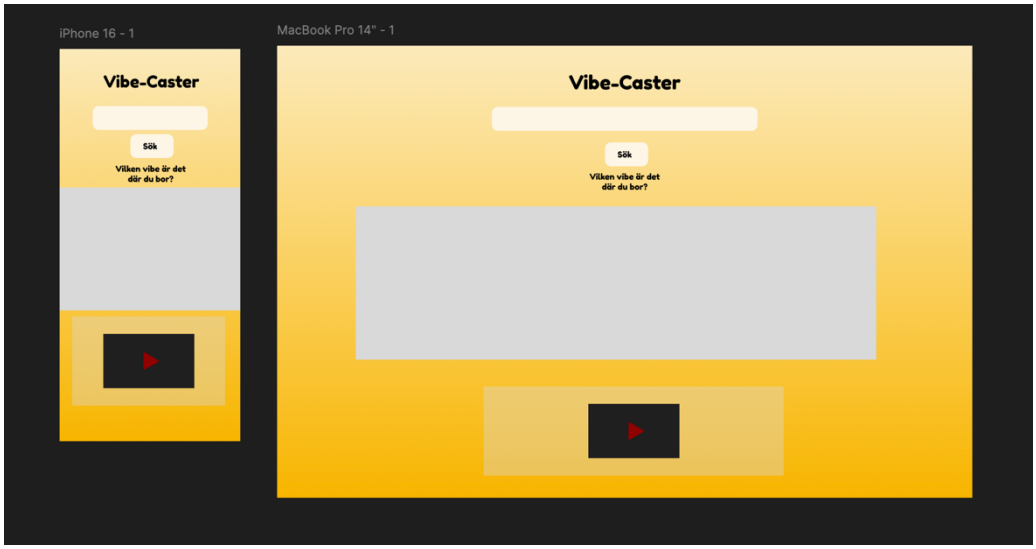


Figure 4.1

För att göra sidan mer levande la jag till en animation i bakgrunden. När JavaScript känner av att det regnar, läggs klassen .is-raining till på sidan vilket sätter igång en CSS-animation som liknar fallande regndroppar.

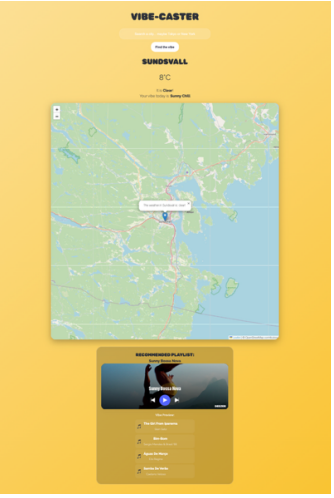
Efter att ha arbetat lite med sidan, kom jag på att det hade varit kul om vädret också styrde bakgrundsfärgen på sidan. Så om man söker på en stad där det är soligt blir bakgrunden varm och gul, men är det molnigt eller regnigt ändras bakgrunden till mörkare gråa och blåa toner. Därmed får man en känsla av vädret, redan innan man hinner läsa vad det står.

6 Resultat

Här tänkte jag visa hur Vibe-Caster blev till slut. När jag väl lyckades få alla delar att arbeta ihop och skapa en ganska kul upplevelse.

6.1 Appens utseende och flöde

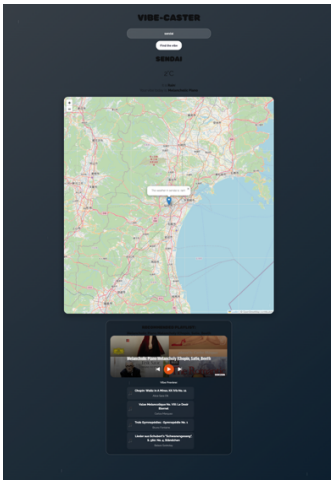
När man först kommer in på sidan möts man av en ren och enkel layout. Jag ville inte att det skulle vara otydligt vad man skulle göra på sidan (se figur 5.1).



Figur 5.1

6.2 Väderstyrda teman och animationer

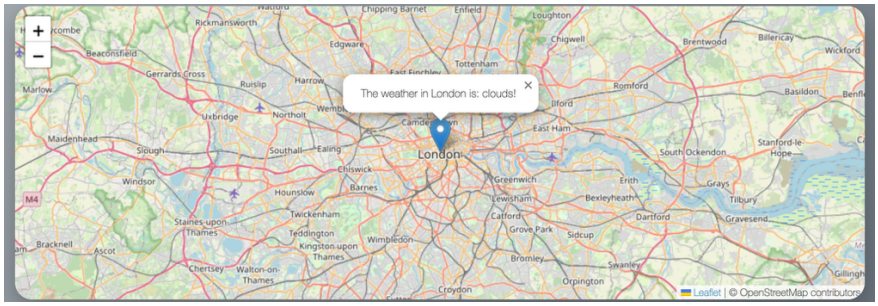
En stor del av resultatet av sidan är hur den ändrar sig efter vädret. I figur 5.1 ovan ser man hur det ser ut när det är soligt, då är det en gradient som övergår från en mörkare gul till en ljusare. I figur 5.3 kan man se hur det ser ut när det regnar. Då ändras bakgrunden till en mörkblå-gradient istället och CSS-regndroppar börjar falla över skärmen. Det här var en av de klurigaste grejerna att få till, men effekten är stor, men syns inte så bra på bilden.



Figur 5.3

6.3 Karta och interaktivitet

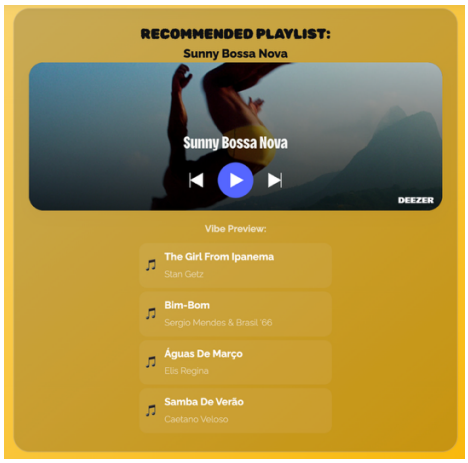
Kartan (Figur 5.4) fungerar som planerat. Den centrerar automatiskt den stad man söker på. Jag la även till en popup-ruta som dyker upp när man har sökt. Den bekräftar namnet på staden och säger vad det är för väderförhållande. Det gör att den känns lite mer integrerad med resten av sidan.



Figur 5.4

6.4 Musikintegration och Preview

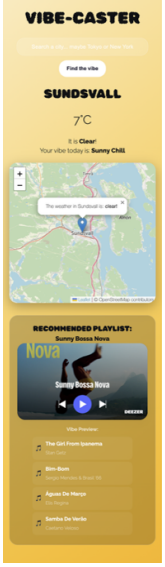
Längst ner på sidan finns resultatet av musiksökningen. Där kan man hitta Deezers musikspelare som spelar musik direkt på sidan. Det finns också en förhandsvisning inför de nästa 4 låtarna som finns i listan (se figur 5.5).



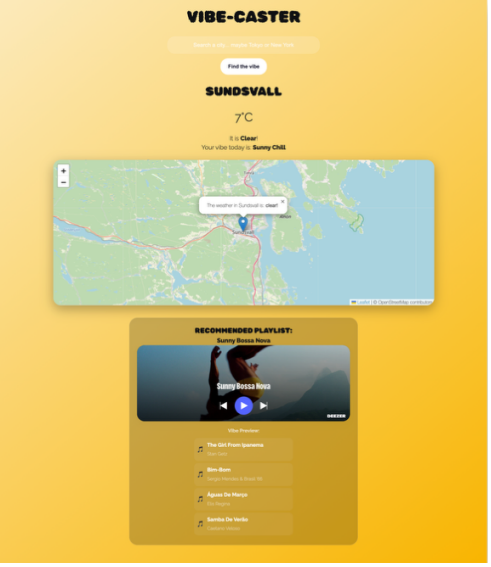
Figur 5.5

6.5 Responsiv design i praktiken

Slutresultatet fungerar lika bra på en liten mobilskärm som på en stor datorskärm. Jag utgick från att sidan skulle funka på mobil, alltså mobile-first strategi. Därför försökte jag till så stor utsträckning som jag kunde använda relativa mått, som procent, vh, rem och så vidare, för att se till att sidans delar var flexibla. Jag la till några max-width för att se till att inte delar av sidan blev för utsträckta på större skärmar, och kunde med det skippa att använda media queries.



Figur 5.6



Figur 5.7

6.6 Teknisk dokumentation

I projektet har jag använt JSDoc. Genom att använda det har JavaScript-filen i projektet blivit väldigt lättöverskådligt. Varje funktion har tydliga beskrivningar och information om vilka parametrar den tar emot.

7 Slutsatser / Analys / Diskussion

Att bygga Vibe-Caster har varit utmanande, men framförallt väldigt roligt. Till att börja med hade jag svårt att veta vilken väg jag skulle ta eftersom det finns så otroligt många olika API:er som verkar roliga att experimentera med.
Mycket statistik, och till exempel Our World in Data verkar ju väldigt intressant att hålla på med.

Däremot tycker jag att det som jag slutligen landade i, att göra en musikspelare som anpassar sig efter vädret på platsen man söker på, passar mig. Lagom lättsamt, men ändå lite givande att få musiktips som jag inte hade hittat själv.

Det svåraste under projektet var nog att lista ut dataflödet mellan de olika API-tjänsterna. Att ett anrop skulle ta värdet från en annan och att få de att hamna i rätt ordning. CORS var också en liten kluring som krävde att jag googlade och testade mig fram en del, men det var intressant att lära mig lite mer om webbsäkerhet och hur det fungerar i praktiken.

När jag analyserar resultatet är jag nog mest nöjd med hur jag lyckades med att bakgrundsfärgen och animationerna ändras beroende på vädret, det gör att det känns lite mer levande än om det bara hade varit en karta och musikspelare.

Hade jag haft mer tid så skulle jag vilja utveckla sidan ännu mer. En idé vore att låta användaren spara sina favoritstäder, eller att sidan läser av besökarens plats automatiskt. Det hade också varit spännande att se om man kunde anpassa musiken efter landet som staden man söker på ligger i. Så till exempel om någon söker på Madrid, så kommer spansk musik och söker man på London så kommer det brittisk musik. Det känns som att då hade man haft möjlighet att utforska andra länders musik på ett roligt sätt.
Jag hade också ändrat på hur rubrikerna ser ut, jag hade velat få till bättre färger, och gärna att färgen på rubriken också ändras med vädret.

Sammanfattningsvis så var det en väldigt rolig och givande uppgift att jobba med. Jag som alltid har beslutsångest känner att det kanske fanns lite för många alternativ som lät roliga och som kanske hade varit bättre såhär i efterhand. Men allt som allt så var det intressant att få arbeta med API:er och att jobba mer med javascript vilket är kul.

Källförteckning

Här följer exempel på hur en källförteckning kan utformas enligt Vancouver-systemet. Den är automatiserad enligt metoden numrerad lista och korsreferenser, som beskrivs i kapitel **Error! Bookmark not defined.**.

[1] Wikipedia, “Mashup (web application hybrid)”, https://en.wikipedia.org/wiki/Mashup_(web_application_hybrid) Hämtad 2026-03-20.

[2] OpenWeatherMap, “Current weather data API”, https://openweathermap.org/current?collection=current_forecast Hämtad 2026-03-20.

[3] Leaflet, “An open-source JavaScript library for mobile-friendly interactive maps”, https://leafletjs.com/ Hämtad 2026-03-20.

[4] Deezer, “Deezer for Developers”, https://developers.deezer.com/login?redirect=/api Hämtad 2026-03-20.

[5] Vite, “Next Generation Frontend Tooling”, https://vite.dev/ Hämtad 2026-03-20.

[6] MDN Web Docs, “Cross-Origin Resource Sharing (CORS)”, https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS Hämtad 2026-03-20.

[7] Sass, “Sass Basics”, https://sass-lang.com/guide/ Hämtad 2026-03-20.

[8] MDN Web Docs, ”How to use promises”, https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Async_JS/Promises Hämtad 2026-03-20.